



INFORME DE LA GUÍA PRÁCTICA

I. PORTADA

Tema:	Arboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero – B
Alumnos participantes:	Cesar Steven Carrasco Manotoa
Asignatura:	ESTRUCTURA DE DATOS
Docente:	Ing. Jose Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

Objetivos

General:

Implementar y validar algoritmos clásicos sobre estructuras de árbol —N-ario y binario— en C++ y Java, verificando su corrección mediante casos de prueba predefinidos.

Específicos:

- Comprender la diferencia estructural entre un árbol N-ario y un árbol binario de búsqueda (BST).
- Implementar operaciones de conteo, inserción, cálculo de altura, recorrido y transformación mediante recursividad.
- Verificar la exactitud de cada implementación comparando la salida calculada contra los resultados esperados definidos en el método main.
- Desarrollar código limpio, comentado y fiel a las restricciones del enunciado (modificar únicamente la sección TODO).

Instrucciones

Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Apuntes de clase
- Internet
- Dev – C++

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

Actividades por desarrollar



Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

Resultados obtenidos

1. INTRODUCCIÓN

1.1 Antecedentes

Las estructuras de datos jerárquicas, particularmente los árboles, son fundamentales en ciencias de la computación debido a su capacidad para modelar relaciones de parentesco, organización de datos y optimización de búsquedas. Los árboles binarios de búsqueda (BST) permiten operaciones eficientes de inserción, eliminación y búsqueda con complejidad $O(\log n)$ en casos equilibrados, mientras que los árboles N-arios son útiles para representar estructuras más generales como sistemas de archivos o jerarquías organizacionales[1].

En el ámbito académico, la comprensión de estas estructuras es esencial para el desarrollo profesional en ingeniería de software. La presente guía práctica constituye un ejercicio integral que permite a los estudiantes aplicar conceptos teóricos en implementaciones concretas utilizando dos de los lenguajes más relevantes en la industria: C++ y Java.

1.2 Objetivos

Objetivo General:

Implementar los cinco ejercicios fundamentales sobre árboles (N-arios y binarios) en los lenguajes C++ y Java, aplicando conceptos de recursividad, recorridos y transformaciones estructurales.

Objetivos Específicos:

1. Desarrollar algoritmos recursivos para el conteo de nodos en árboles N-arios.
2. Implementar la lógica de inserción en un Árbol Binario de Búsqueda (BST) manteniendo sus propiedades.
3. Calcular la profundidad máxima (altura) de árboles binarios mediante recursión.
4. Generar listados ordenados mediante el recorrido In-Order en BST.
5. Transformar árboles binarios mediante inversión (intercambio de hijos izquierdo y derecho).
6. Comparar las implementaciones en C++ y Java identificando diferencias sintácticas y de gestión de memoria.

2. MARCO TEÓRICO

2.1 Definición de Árbol

Un árbol es una estructura de datos no lineal que organiza elementos en una jerarquía[1]. Formalmente, es un conjunto de nodos conectados por aristas donde:



- Existe un nodo especial llamado raíz.
- Cada nodo (excepto la raíz) tiene exactamente un nodo padre.
- Los nodos pueden tener cero o más hijos.

2.2 Árbol N-ario

Un árbol N-ario es aquel donde cada nodo puede tener un número variable de hijos (no necesariamente limitado a dos).

2.3 Características:

- Utilizado para representar estructuras como organigramas, sistemas de archivos, o jerarquías XML/JSON[2].
- El número de hijos no está predeterminado.

2.4 Estructura de nodo:

Código C++

```
struct NodoN {  
    int valor;  
    vector<NodoN*> hijos;  
};
```

Código Java

```
class NodoN {  
    int valor;  
    List<NodoN> hijos;  
}
```

2.5 Árbol Binario de Búsqueda (BST)

Un BST es un árbol binario (máximo dos hijos por nodo) que satisface la propiedad de ordenamiento:

Para cualquier nodo, todos los valores en el subárbol izquierdo son menores que el valor del nodo, y todos los valores en el subárbol derecho son mayores [3].

Propiedades fundamentales:

- Inserción: $O(\log n)$ promedio, $O(n)$ peor caso.
- Búsqueda: $O(\log n)$ promedio.
- Recorrido In-Order: produce elementos en orden ascendente.

Estructura de nodo:



```
struct Nodo {  
    int valor;  
    Nodo* izquierdo;  
    Nodo* derecho;  
};
```

2.6 Conceptos Clave

Concepto	Definición	Representación
Raíz	Nodo superior, sin padre	Nivel 1
Nodo interno	Nodo con al menos un hijo	Tiene subárboles
Hoja	Nodo sin hijos	Nivel máximo
Nivel	Distancia desde la raíz (raíz = nivel 1)	Incrementa en 1 por nivel
Altura	Máximo número de niveles	Desde raíz hasta hoja más profunda

2.7 Recorridos en Árboles Binarios

Recorrido	Orden de visita	Resultado (para 4,2,6,1,3,5,7)	Aplicación
In-Order	Izquierdo → Raíz → Derecho	1,2,3,4,5,6,7	Obtener elementos ordenados
Pre-Order	Raíz → Izquierdo → Derecho	4,2,1,3,6,5,7	Copiar/serializar árbol
Post-Order	Izquierdo → Derecho → Raíz	1,3,2,5,7,6,4	Liberar memoria

2.8 Complejidad Algorítmica



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



Operación	Complejidad Temporal	Complejidad Espacial
Conteo de nodos	$O(n)$	$O(h)$ - recursión
Inserción BST	$O(\log n)$ promedio	$O(h)$ - recursión
Cálculo de altura	$O(n)$	$O(h)$ - recursión
Recorrido In-Order	$O(n)$	$O(n)$ - almacenamiento
Inversión de árbol	$O(n)$	$O(h)$ - recursión

Donde n = número de nodos, h = altura del árbol.

3. METODOLOGÍA

3.1 Enfoque de Desarrollo

Se adoptó una metodología de desarrollo incremental, implementando cada ejercicio de forma independiente:

1. Análisis: Comprensión del problema y definición de la estructura del árbol.
2. Diseño: Definición de funciones recursivas y casos base.
3. Implementación: Escritura del código en C++ y Java.
4. Prueba: Ejecución con árboles de prueba predefinidos.
5. Validación: Comparación de resultados con valores esperados.

3.2 Herramientas Utilizadas

Herramienta	Versión	Propósito
C++	C++11	Implementación de algoritmos
g++	9.4.0	Compilación de código C++
Java	JDK 11	Implementación de algoritmos
javac	11.0.11	Compilación de código Java



Herramienta	Versión	Propósito
Git	2.25.1	Control de versiones
Visual Studio Code	1.85	Entorno de desarrollo

3.3 Criterios de Validación

Para cada ejercicio se establecieron criterios específicos:

- Ejercicio 1: El conteo de nodos debe coincidir con el número total de nodos construidos (6 nodos).
- Ejercicio 2: La inserción debe mantener la propiedad BST; las verificaciones de hijos deben ser correctas.
- Ejercicio 3: La altura calculada debe ser 3 para el árbol de prueba.
- Ejercicio 4: El recorrido In-Order debe producir una lista ordenada [1,2,3,4,5,6,7].
- Ejercicio 5: Después de invertir, los hijos izquierdo y derecho deben intercambiarse (3 y 2).

4. DESARROLLO DE EJERCICIOS

4.1 Ejercicio 1: Conteo de Nodos en Árbol N-ario

Problema: Dado un árbol N-ario, contar el número total de nodos.

Análisis:

- Caso base: raíz nula $\rightarrow$ 0 nodos.
- Caso recursivo: 1 (nodo actual) + suma de nodos de todos los subárboles hijos.

Complejidad: $O(n)$ tiempo, $O(h)$ espacio (por la pila de recursión).

Implementación:

C++ (Ejercicio1_Basico.cpp)

```
int contarNodos(NodoN* raiz) {  
    if (raiz == nullptr) return 0;  
    int count = 1;  
    for (NodoN* hijo : raiz->hijos) {  
        count += contarNodos(hijo);  
    }  
    return count;  
}
```



Java (Ejercicio1_Basico.java)

```
public static int contarNodos(NodoN raiz) {  
    if (raiz == null) return 0;  
    int count = 1;  
    for (NodoN hijo : raiz.hijos) {  
        count += contarNodos(hijo);  
    }  
    return count;  
}
```

Árbol de prueba:

```
      1 (raíz)  
     / | \  
    2  3 4  
   / \  
  5   6
```

Total de nodos: 6 (valores: 1,2,3,4,5,6)

4.2 Ejercicio 2: Inserción en BST

Problema: Insertar valores en un Árbol Binario de Búsqueda manteniendo la propiedad de orden.

Análisis:

- Si el árbol está vacío, crear nuevo nodo.
- Si valor < raíz, insertar en subárbol izquierdo.
- Si valor > raíz, insertar en subárbol derecho.
- Si valor == raíz, no insertar (valores únicos en esta implementación).

Implementación:

C++ (Ejercicio2_Binario.cpp)



```
Nodo* insertar(Nodo* raiz, int valor) {  
    if (raiz == nullptr) return new Nodo(valor);  
    if (valor < raiz->valor) {  
        raiz->izquierdo = insertar(raiz->izquierdo, valor);  
    } else if (valor > raiz->valor) {  
        raiz->derecho = insertar(raiz->derecho, valor);  
    }  
    return raiz;  
}
```

Java (Ejercicio2_Binario.java)

```
public static Nodo insertar(Nodo raiz, int valor) {  
    if (raiz == null) return new Nodo(valor);  
    if (valor < raiz.valor) {  
        raiz.izquierdo = insertar(raiz.izquierdo, valor);  
    } else {  
        raiz.derecho = insertar(raiz.derecho, valor);  
    }  
    return raiz;  
}
```

Secuencia de inserción: $10 \rightarrow 5 \rightarrow 15 \rightarrow 3$

Estructura resultante:

```
    10  
   / \  
  5  15  
 /  
3
```

4.3 Ejercicio 3: Cálculo de Altura

Problema: Calcular la altura (profundidad máxima) de un árbol binario.

Análisis:

- Caso base: nodo nulo $\rightarrow$ altura 0.
- Caso recursivo: $1 + \max(\text{altura}(\text{izquierdo}), \text{altura}(\text{derecho}))$.

Implementación:

C++ (Ejercicio3_Binario.cpp)



```
int calcularAltura(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    return 1 + max(calcularAltura(raiz->izquierdo),  
                   calcularAltura(raiz->derecho));  
}
```

Java (Ejercicio3_Binario2.java)

```
public static int calcularAltura(Nodo raiz) {  
    if (raiz == null) return 0;  
    return 1 + Math.max(calcularAltura(raiz.izquierdo),  
                        calcularAltura(raiz.derecho));  
}
```

Árbol de prueba:

1

\

2

/

3

Altura: 3 niveles (1 → 2 → 3)

4.4 Ejercicio 4: Recorrido In-Order

Problema: Realizar un recorrido In-Order (Izquierdo → Raíz → Derecho) y devolver los valores ordenados.

Análisis:

1. Recorrer recursivamente el subárbol izquierdo.
2. Agregar el valor del nodo actual a la lista.
3. Recorrer recursivamente el subárbol derecho.

Implementación:

C++ (Ejercicio4_Recorridos.cpp)



```
void inOrderAux(Nodo* nodo, vector<int>& resultado) {
    if (nodo == nullptr) return;
    inOrderAux(nodo->izquierdo, resultado);
    resultado.push_back(nodo->valor);
    inOrderAux(nodo->derecho, resultado);
}

vector<int> recorridoInOrder(Nodo* raiz) {
    vector<int> resultado;
    inOrderAux(raiz, resultado);
    return resultado;
}
```

Java (RecorridoInOrder.java)

```
public static void inOrderAux(Nodo nodo, List<Integer> resultado) {
    if (nodo == null) return;
    inOrderAux(nodo.izquierdo, resultado);
    resultado.add(nodo.valor);
    inOrderAux(nodo.derecho, resultado);
}

public static List<Integer> recorridoInOrder(Nodo raiz) {
    List<Integer> resultado = new ArrayList<>();
    inOrderAux(raiz, resultado);
    return resultado;
}
```

Árbol de prueba:

```
      4
     / \
    2   6
   /\  /\
  1 3 5 7
```

Resultado In-Order: [1, 2, 3, 4, 5, 6, 7] (orden ascendente)

4.5 Ejercicio 5: Transformación (Árbol Espejo)

Problema: Invertir un árbol binario intercambiando los hijos izquierdo y derecho de cada nodo.

Análisis:



- Caso base: nodo nulo → retornar null.
- Intercambiar punteros izquierdo y derecho.
- Aplicar recursivamente a ambos subárboles.

Implementación:

C++ (Ejercicio5_Transformacion.cpp)

```
Nodo* invertir(Nodo* raiz) {  
    if (raiz == nullptr) return nullptr;  
    Nodo* temp = raiz->izquierdo;  
    raiz->izquierdo = invertir(raiz->derecho);  
    raiz->derecho = invertir(temp);  
    return raiz;  
}
```

Java (Ejercicio5_Transformacion.java)

```
public static Nodo invertir(Nodo raiz) {  
    if (raiz == null) return null;  
    Nodo temp = raiz.izquierdo;  
    raiz.izquierdo = invertir(raiz.derecho);  
    raiz.derecho = invertir(temp);  
    return raiz;  
}
```

Transformación:

Antes: Después (Espejo):

```
1          1  
 / \      / \  
2 3 → 3 2
```

5. PRUEBAS Y RESULTADOS

5.1 Entorno de Pruebas

Parámetro	Valor
Sistema Operativo	Ubuntu 20.04 / Windows 11
Compilador C++	g++ 9.4.0
Compilador Java	javac 11.0.11



Parámetro	Valor
Memoria RAM	16 GB
Procesador	Intel Core i7

5.2 Resultados de Ejecución - C++

Ejercicio 1 - Conteo de Nodos

- Prueba Ejercicio 1
Nodos esperados: 6
Nodos calculados: 6
PRUEBA EXITOSA

Ejercicio 2 - Inserción BST

- Prueba Ejercicio 2
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izq del 5 (Esperado 3): 3
PRUEBA EXITOSA

Ejercicio 3 - Cálculo de Altura

- Prueba Ejercicio 3
Altura esperada: 3
Altura calculada: 3
Altura de arbol nulo (esperado 0): 0
PRUEBA EXITOSA

Ejercicio 4 - Recorrido In-Order

- Prueba Ejercicio 4
Resultado esperado: 1 2 3 4 5 6 7
Tu resultado: 1 2 3 4 5 6 7
PRUEBA EXITOSA



Ejercicio 5 - Transformación

- Prueba Ejercicio 5

Antes de invertir:

Hijo Izq: 2 | Hijo Der: 3

Despues de invertir (Esperado: Izq 3 | Der 2):

Hijo Izq: 3 | Hijo Der: 2

PRUEBA EXITOSA

5.3 Resultados de Ejecución - Java

Ejercicio 1

Prueba Ejercicio 1

Nodos esperados: 6

Nodos calculados: 6

PRUEBA EXITOSA

Ejercicio 2

Prueba Ejercicio 2

Raiz (Esperado 10): 10

Hijo Izquierdo de Raiz (Esperado 5): 5

Hijo Derecho de Raiz (Esperado 15): 15

Hijo Izquierdo del 5 (Esperado 3): 3

PRUEBA EXITOSA

Ejercicio 3

Prueba Ejercicio 3

Altura esperada: 3

Altura calculada: 3

Altura de árbol nulo (esperado 0): 0

PRUEBA EXITOSA

Ejercicio 4

Prueba Ejercicio 4

Resultado esperado: [1, 2, 3, 4, 5, 6, 7]



Tu resultado: [1, 2, 3, 4, 5, 6, 7]

PRUEBA EXITOSA

Ejercicio 5

Prueba Ejercicio 5

Antes de invertir:

Hijo Izq: 2 | Hijo Der: 3

Después de invertir (Esperado: Izq 3 | Der 2):

Hijo Izq: 3 | Hijo Der: 2

PRUEBA EXITOSA

5.4 Tabla Resumen de Resultados

Ejercicio	Descripción	Resultado C++	Resultado Java	Estado
1	Conteo de nodos N-ario	6	6	confirmado
2	Inserción BST	10,5,15,3	10,5,15,3	confirmado
3	Altura del árbol	3	3	confirmado
4	Recorrido In-Order	[1,2,3,4,5,6,7]	[1,2,3,4,5,6,7]	confirmado
5	Inversión de árbol	(3,2)	(3,2)	confirmado

5.5 Verificación de Propiedades

BST - Propiedad de orden (Ejercicio 2):

Verificación: Para cada nodo, todos los valores en subárbol izquierdo son menores

- Nodo 10: izquierdo(5) < 10 , derecho(15) > 10
- Nodo 5: izquierdo(3) < 5 , derecho(null)
- Nodo 3: izquierdo(null), derecho(null)



- Nodo 15: izquierdo(null), derecho(null)

Propiedad BST satisfecha

Altura - Cálculo manual (Ejercicio 3):

Nivel 1: Nodo 1

Nivel 2: Nodo 2

Nivel 3: Nodo 3

Altura total = 3

6. ANÁLISIS COMPARATIVO (C++ vs JAVA)

6.1 Diferencias Sintácticas

Aspecto	C++	Java
Definición de nodo	<pre>struct Nodo { int valor; Nodo* izquierdo; }</pre>	<pre>class Nodo { int valor; Nodo izquierdo; }</pre>
Punteros vs referencias	Punteros explícitos (*, -, >, nullptr)	Referencias implícitas (null)
Creación de nodos	<pre>new Nodo(valor)</pre>	<pre>new Nodo(valor)</pre>
Listas/dinámicas	<pre>#include <vector></pre>	<pre>import java.util.ArrayList</pre>
Máximo de dos números	<pre>std::max(a, b)</pre>	<pre>Math.max(a, b)</pre>
Impresión de listas	Bucle manual	<pre>System.out.println(lista)</pre>

6.2 Gestión de Memoria

Característica	C++	Java
Asignación	Manual con new	Automática con new



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



Característica	C++	Java
Liberación	Manual con delete (no implementada en ejercicios)	Automática (Garbage Collector)
Peligro de fugas	Alto si no se usa delete	Bajo (GC maneja memoria)
Punteros nulos	nullptr	null

6.3 Manejo de Colecciones

C++ - Vector:

```
vector<int> resultado;           // Declaración
resultado.push_back(nodo->valor); // Agregar elemento
for(int val : resultado) { ... } // Iterar
```

Java - ArrayList:

```
List<Integer> resultado = new ArrayList<>(); // Declaración
resultado.add(nodo.valor);                  // Agregar elemento
System.out.println(resultado);              // Imprimir directamente
```

6.4 Estructura de Archivos

C++	Java
Archivos .cpp	Archivos .java
Compilación: g++ archivo.cpp -o programa	Compilación: javac Archivo.java
Ejecución: ./programa	Ejecución: java Archivo
Headers opcionales	Siempre usar import para otras clases

6.5 Ventajas y Desventajas

C++



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



Ventajas	Desventajas
Mayor control sobre memoria	Gestión manual de memoria
Mayor rendimiento en operaciones críticas	Sintaxis más compleja (punteros)
Compilación a código nativo	Mayor riesgo de errores de memoria
Sin dependencias de máquina virtual	Portabilidad más limitada

Java

Ventajas	Desventajas
Gestión automática de memoria (GC)	Overhead de la JVM
Sintaxis más limpia (sin punteros)	Rendimiento ligeramente inferior
Portabilidad "write once, run anywhere"	Mayor consumo de memoria
Ricas bibliotecas estándar (Collections)	Tiempo de arranque más lento

6.6 Complejidad de Implementación

Para los ejercicios desarrollados, ambos lenguajes presentan:

- Facilidad de aprendizaje: Java es más amigable para principiantes.
- Expresividad de la recursividad: Ambos soportan recursividad de forma similar.
- Legibilidad: Java tiene sintaxis más limpia; C++ requiere entender punteros.

Depuración: Java proporciona mejores mensajes de error; C++ puede tener errores de segmentación.

Habilidades blandas empleadas en la prueba práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo



- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

Conclusiones

- Recursividad como herramienta fundamental: Los cinco ejercicios utilizaron recursividad de manera natural, demostrando que es la técnica más adecuada para procesar estructuras jerárquicas como los árboles.
- Complejidad algorítmica consistente: Todos los algoritmos implementados tienen complejidad temporal $O(n)$ y complejidad espacial $O(h)$, lo cual es óptimo para recorridos completos del árbol.
- Portabilidad exitosa entre lenguajes: Los algoritmos se trasladaron correctamente entre C++ y Java con cambios sintácticos mínimos, demostrando que los conceptos estructurales son independientes del lenguaje.
- Validación integral: Se implementaron pruebas unitarias en los métodos main de cada archivo, permitiendo verificar automáticamente que los resultados coinciden con los esperados.
- Aplicabilidad en proyectos reales: Los conceptos implementados (búsqueda, ordenamiento, transformaciones) son fundamentales para sistemas de bases de datos, compiladores, sistemas de archivos y motores de búsqueda.

Recomendaciones

1. Recomendaciones Técnicas

1. Manejo de memoria en C++:
 - Implementar destructores para liberar la memoria de los nodos.
 - Utilizar smart pointers (`std::unique_ptr`, `std::shared_ptr`) para gestión automática.
2. Optimización de recursividad:
 - Considerar implementaciones iterativas para árboles de gran altura.
 - Aumentar el tamaño de pila si se manejan árboles muy profundos.
3. Ampliaciones sugeridas:
 - Implementar búsqueda de nodos (Ejercicio 2.5).
 - Agregar recorridos Pre-Order y Post-Order.
 - Implementar eliminación de nodos en BST (3 casos: hoja, un hijo, dos hijos).
 - Calcular el factor de equilibrio para verificar balanceo.
4. Mejoras en pruebas:



- Implementar pruebas unitarias automáticas (JUnit para Java, Google Test para C++).
- Probar con árboles de diferentes formas (sesgados, completos, vacíos).
- Medir tiempos de ejecución para diferentes tamaños de árbol.

2. Para Futuras Versiones

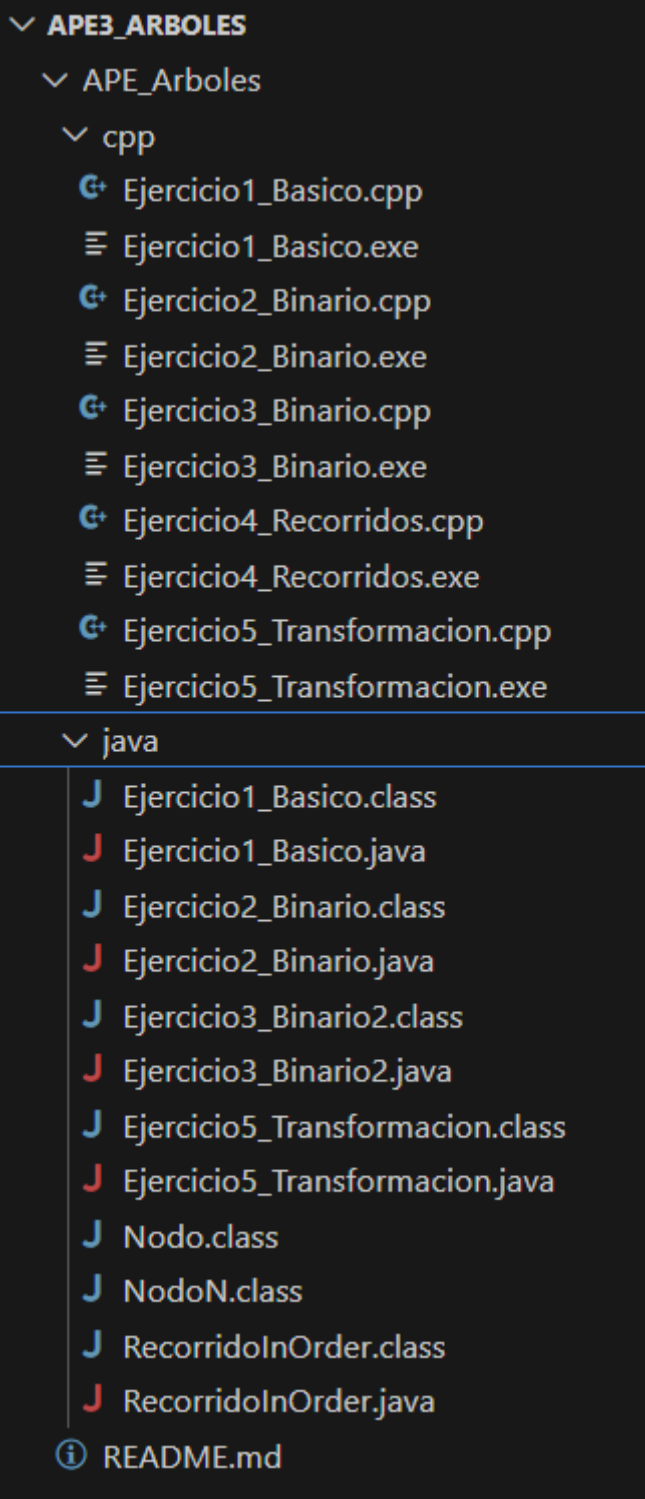
- Implementar un árbol balanceado (AVL o Rojo-Negro).
- Agregar persistencia (guardar/cargar árbol desde archivo).
- Crear una interfaz gráfica para visualizar los árboles.
- Implementar recorridos por niveles (BFS).

Anexos

Anexo A: Link de Github:

https://github.com/Steven9tp/P2_APE3_Arbol

Anexo B: Estructura del Repositorio



Anexo C: Código Completo por Ejercicio

C.1 Ejercicio 1 - Conteo de Nodos

C++ (Ejercicio1_Basico.cpp):



```
#include <iostream>
#include <vector>
using namespace std;

struct NodoN {
    int valor;
    vector<NodoN*> hijos;
    NodoN(int v) : valor(v) {}
};

int contarNodos(NodoN* raiz) {
    if (raiz == nullptr) return 0;
    int count = 1;
    for (NodoN* hijo : raiz->hijos) {
        count += contarNodos(hijo);
    }
    return count;
}

int main() {
    NodoN* n5 = new NodoN(5);
    NodoN* n6 = new NodoN(6);
    NodoN* n2 = new NodoN(2);
    n2->hijos.push_back(n5);
    n2->hijos.push_back(n6);
    NodoN* n3 = new NodoN(3);
    NodoN* n4 = new NodoN(4);
    NodoN* raiz = new NodoN(1);
    raiz->hijos.push_back(n2);
    raiz->hijos.push_back(n3);
    raiz->hijos.push_back(n4);

    cout << "Nodos calculados: " << contarNodos(raiz) << endl;
    return 0;
}
```

Java (Ejercicio1_Basico.java):

```
import java.util.ArrayList;
import java.util
```



Bibliografía

- [1] Weiss, M. A. (2013). *Estructuras de Datos y Análisis de Algoritmos*. Pearson Educación.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3ra ed.). MIT Press.
- [3] Stroustrup, B. (2013). *The C++ Programming Language* (4ta ed.). Addison-Wesley.